

Concepts of Hashing

Collision Resolution Strategies via Open Addressing

1. Introduction to Hashing & Collisions

Hashing is a technique used to map data of arbitrary size to fixed-size values. In a **Hash Table**, a hash function calculates an index into an array of slots, from which the desired value can be found.

A **collision** occurs when the hash function maps two different keys to the exact same index in the array. Since two values cannot occupy the same slot, a resolution strategy is required.

2. Open Addressing

Open addressing resolves hash table collisions by finding the next available slot within the array itself, completely avoiding external data structures like linked lists (as seen in separate chaining). When a collision happens, the algorithm systematically *probes* the array until it finds an empty space.

There are three primary probing strategies:

2.1 Linear Probing

Linear probing is the simplest approach. If the initially calculated slot is full, the algorithm checks the very next slot (wrapping around to the beginning if it reaches the end of the array).

Linear Probing Formula

$$H(k, i) = (h(k) + i) \bmod M$$

Where **k** is the key, **i** is the attempt number (0, 1, 2, ...), and **M** is the table size.

The Drawback: Primary Clustering.

If slots 3, 4, and 5 are filled, any key hashing to 3, 4, or 5 will collide and ultimately fill slot 6. These contiguous blocks merge and snowball, degrading search and insertion performance from $O(1)$ to $O(n)$.

2.2 Quadratic Probing

To spread out the probes and prevent primary clustering, quadratic probing increases the step size quadratically on each attempt (+1, +4, +9, +16...).

Quadratic Probing Formula

$$H(k, i) = (h(k) + i^2) \bmod M$$

The Drawback: Secondary Clustering.

While it fixes contiguous primary clusters, any keys that hash to the same initial index will follow the exact same sequence of jumps, creating a milder form of clustering.

Java Implementation: Quadratic Probing

```
public void insertQuadratic(int[] table, int key) {
    int size = table.length;
    int hash = key % size;
    int i = 0;

    // Continue probing until an empty slot (assumed 0) is found
    while (table[(hash + (i * i)) % size] != 0) {
        i++;
        if (i >= size) {
            System.out.println("Table is full or insertion failed");
            return;
        }
    }
    table[(hash + (i * i)) % size] = key;
}
```

2.3 Double Hashing

Double hashing is the most robust open addressing method. It eliminates both primary and secondary clustering by using a *second hash function* to determine the step size. Because the step size is tied to the key itself, keys that hash to the same initial index will jump by completely different amounts.

Double Hashing Formula

$$H(k, i) = (h_1(k) + i \times h_2(k)) \bmod M$$

Understanding the Second Hash Function (h_2)

The second hash function usually takes the following form:

$$h_2(k) = R - (k \bmod R)$$

What is R ? R is a prime number strictly smaller than the table size M . For example, if $M = 11$, a good choice for R is 7.

Why this specific formula?

- **It never returns zero:** If the step size evaluates to 0, the algorithm enters an infinite loop checking the same slot. $k \bmod 7$ returns values from 0 to 6. Subtracting that from 7 ensures the step size is always safely between 1 and 7.
- **Guarantees visiting every slot:** For the probing sequence to eventually check every slot without skipping, the step size and the table size must be *co-prime*. Using a prime number R smaller than a prime table size M guarantees this mathematical property.

Java Implementation: Double Hashing

```
public void insertDoubleHashing(int[] table, int key) {
    int size = table.length;

    // First hash determines the initial index
    int hash1 = key % size;

    // R is a deliberately chosen prime number smaller than 'size'
    int primeR = 7;
    // Second hash determines the step size
    int hash2 = primeR - (key % primeR);

    int i = 0;
    while (table[(hash1 + i * hash2) % size] != 0) {
        i++;
        if (i >= size) {
            System.out.println("Table is full");
            return;
        }
    }
    table[(hash1 + i * hash2) % size] = key;
}
```